MOBILE APPLICATION PENETRATION TESTING

From Static Analysis to Runtime Exploitation

Agenda & Timing (3 Hours)

1	**Modules 1–3**	Threat landscape, MASVS/MASTG, Android attack surface, lab & toolchain setup
2	**Modules 4–5**	Static analysis methodology and dynamic analysis methodology
3	**Live Demo**	Crack OWASP UnCrackable App Level 1 — static route AND dynamic route
4	**Module 6**	Vulnerability tour, ethics & legal, key takeaways, Q&A

BY THE END OF THIS LECTURE, YOU WILL BE ABLE TO

Learning Objectives

- Explain how the OWASP MASVS and MASTG structure a mobile penetration test
- Map an Android app's attack surface: components, IPC, storage, network, binary
- Perform static analysis: decompile an APK and spot hardcoded secrets & weak crypto
- Perform dynamic analysis: intercept traffic and instrument a running app with Frida/Objection
- Solve OWASP UnCrackable App Level 1 using both a static and a dynamic approach
- Write findings that map cleanly to MASVS control IDs



Prerequisite check

You already know: HTTP, Burp Suite, common web vulns (OWASP Top 10), basic reverse engineering concepts. Today we port that mindset onto a compiled, sandboxed mobile client.

MODULE 1

Why Mobile Is a Different Battlefield

Threat landscape, MASVS/MASTG, and what changes when the client is compiled

Web Pentest Skills vs. the Mobile Attack Surface

What carries over from web

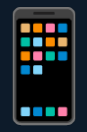
- Intercepting proxy workflow (Burp/mitmproxy)
- Server-side logic flaws: auth, IDOR, injection
- Session & token handling weaknesses
- Business-logic abuse and input validation

What's new on mobile

- The client itself is reversible compiled code
- Sensitive data can persist on the device (storage)
- Inter-process communication is a first-class attack surface
- The OS actively fights you: sandboxing, signing, pinning
- You must control a device/emulator, not just a browser

The Mobile Threat Landscape

- **Client-side trust is misplaced.** Business logic and secrets baked into the APK/IPA are extractable by anyone with the file.
- **Supply-chain & third-party SDKs.** Ad SDKs, analytics, and outdated libraries often leak data or open new attack surface.
- **Repackaging & piracy.** Attackers patch out license checks, ads, or root/jailbreak detection and redistribute the app.
- **Device diversity.** Rooted phones, custom ROMs, and emulators break assumptions the app makes about its environment.



Mobile Top 10

OWASP maintains a Mobile Application Security Top 10, structurally similar to the Web Top 10 but reordered around storage, crypto, auth, network and platform misuse.

THE CHECKLIST WE TEST AGAINST ALL DAY

OWASP MASVS: The Verification Standard



V1 · Architecture

Threat modeling, secure SDLC, dependency management



V2 · Storage

What the app persists, where, and how well it's protected



V3 · Cryptography

Algorithm choice, key management, randomness



V4 · Authentication

Login, session, biometric & local auth handling



V5 · Network

TLS configuration and certificate/public-key pinning



V6 · Platform

IPC, WebViews, deep links, permissions misuse



V7 · Code Quality

Hardcoded secrets, memory safety, build hardening



V8 · Resiliency

Anti-tampering, root/jailbreak & debug detection

OWASP MASTG & the Crackme Series

- MASTG (Mobile Application Security Testing Guide) gives step-by-step test cases for every MASVS control — think of it as the 'how' to MASVS's 'what'
- The OWASP MAS project ships deliberately vulnerable 'Crackme' apps so students can practice without breaking real, third-party software
- UnCrackable App for Android — Level 1: hides a secret string behind weak crypto + basic root/debug detection (our demo today)
- Levels 2–4 progressively add native code, stronger obfuscation, and anti-Frida tricks — great follow-up homework



Today's target

UnCrackable-Level1.apk

Goal: recover the hidden secret string the app is trying to hide from you — twice, two different ways.

MODULE 2

Android Attack Surface

What actually ships inside an APK, and where it can be attacked

Anatomy of an APK



AndroidManifest.xml

Components, permissions, exported flags, deep-link intent filters



classes.dex

Compiled Dalvik/ART bytecode — reversible with jadx/apktool



resources.arsc + res/

Strings, layouts, sometimes hardcoded keys or endpoints



lib/<abi>/*.so

Native code (JNI) — needs Ghidra/IDA, harder to reverse



META-INF/

Signing certificate & manifest hashes — integrity check surface



assets/

Bundled files: configs, ML models, sometimes embedded secrets

EVERY ONE OF THESE IS A POTENTIAL ENTRY POINT

App Components & the IPC Surface



Activities

UI screens — exported=true means other apps can launch them directly



Services

Background work — can expose privileged operations via Binder/AIDL



Broadcast Receivers

React to system/app events — unprotected receivers can be triggered externally



Content Providers

Structured data access — classic source of SQL injection & data leaks



Intents & Deep Links

Inter-app messaging — implicit intents can be intercepted or spoofed



WebViews

Embedded browser — JS bridges (addJavascriptInterface) are high-risk

A One-Slide Look at iOS

Android (today's focus)

- APK = ZIP of DEX bytecode + manifest + resources
- jadx / apktool decompile close to source
- Frida + rooted emulator for full dynamic control
- Root detection is the main runtime defense

iOS (for contrast)

- IPA = ZIP of a Mach-O binary + Info.plist + bundle
- class-dump / Hopper / IDA for static analysis
- Frida + jailbroken device (or on-device jailbreak-free tricks)
- Keychain, App Transport Security, code signing are core controls

MODULE 3

Building the Test Lab

Standing up an environment you fully control before you touch a target

Lab Setup Checklist

1	**Rooted emulator or device**	Android Studio AVD with a rooted system image, or Genymotion, or Magisk on a real test device
2	**Frida server on-device**	Match frida-server's version exactly to your host frida-tools version — mismatches fail silently
3	**Proxy configured & trusted**	Burp Suite / mitmproxy running; its CA certificate installed as a device-trusted root
4	**Target APK staged**	UnCrackable-Level1.apk downloaded from the OWASP MAS Crackmes repository
5	**Connectivity sanity check**	`adb devices` shows the emulator; `frida-ps -U` lists running processes

ONE TOOL PER JOB, USED TOGETHER

The Toolchain Tour



ADB

Android Debug Bridge — shell access, install/uninstall, log streaming



jadx / apktool

Decompile DEX to readable Java, or disassemble to editable smali



MobSF

Automated static + dynamic scanning; fast first-pass triage



Frida

Runtime instrumentation — hook and rewrite behavior of a live app



Objection

Frida-powered CLI: root/SSL-pinning bypass recipes without writing JS



Burp Suite / mitmproxy

Intercept, inspect and modify the app's network traffic



drozer

Enumerate and attack exported Android IPC components



apkleaks / TruffleHog

Automated regex/entropy scanning for hardcoded secrets

MODULE 4

Static Analysis Methodology

Reading the app without ever running it

NO EMULATOR REQUIRED FOR ANY OF THIS

Static Analysis Workflow

1	**Decompile**	jadx-gui app.apk (Java-like source) and/or apktool d app.apk (editable smali + resources)
2	**Manifest review**	Exported components, permissions requested, debuggable flag, allowBackup, deep links
3	**Secret & key hunting**	grep for API keys, hardcoded passwords, private keys, and suspicious Base64 blobs
4	**Crypto review**	Look for ECB mode, static IVs, home-grown algorithms, or keys derived from constants
5	**Automated triage**	Run MobSF's static scan in parallel to catch what manual review might miss

What Static Analysis Reveals

A pattern you will see constantly: secrets baked directly into decompiled source.

```
# jadx-gui: sg/vantagepoint/uncrackable1/CodeCheck.java (illustrative)

public class CodeCheck {
    public boolean check_pin(String pin) {
        byte[] enc = Base64.decode("8ClmM+aX3AceHWyz1Rl/8A==", 0);
        byte[] key = "8d127684cbc37c17".getBytes();      // <-- hardcoded key
        String secret = AESDecryptor.decrypt(enc, key);   // ECB mode, no IV
        return pin.equals(secret);
    }
}

# Takeaway: the key AND the ciphertext are visible to anyone with the APK.
# We do not need to run the app at all to recover the logic — see next slide.
```

MODULE 5

Dynamic Analysis Methodology

Watching — and steering — the app while it runs

Dynamic Analysis Workflow

1. **Traffic interception** — Route the device through Burp/mitmproxy; install the proxy CA as a trusted root
2. **Defeat certificate pinning** — Frida script or Objection recipe to disable the app's TrustManager/pinning checks
3. **Runtime instrumentation** — Attach Frida, enumerate loaded classes, hook methods of interest with Interceptor/Java.perform
4. **Defense bypass** — Neutralize root detection, debugger detection, and integrity/tamper checks that block testing
5. **IPC probing** — Use drozer to invoke exported activities/providers/receivers directly, outside the intended UI flow

Frida & Objection Essentials



Java.perform()

Entry point for hooking managed (Java/Kotlin) code from a Frida script



Interceptor.attach()

Hook native functions; inspect or rewrite arguments and return values



Java.use(cls)

Get a live handle to a class to override method implementations at runtime



frida -U -f <pkg> -l x.js

Spawn the app with a hook script attached before its own code runs



objection explore

Interactive shell: android root disable, sslpinning disable, hooking watch



hooking watch class_method

Log every call, argument and return value for a chosen method — zero scripting

Live Demo: OWASP UnCrackable App — Level 1

- The app asks the user to enter a secret string in a text field
- If it detects a rooted device, it shows a toast and force-closes — before you can even try
- A hidden secret value is compared against whatever the user typed
- Our goal: recover that secret value, two independent ways



Two winning conditions

Path A — Static: never run the app; read the decompiled code and decrypt offline.

Path B — Dynamic: run the app, bypass its defenses, and watch it reveal the secret at runtime.

Demo — Path A: Static Reverse Engineering

1. **Decompile** — jadx-gui UnCrackable-Level1.apk — browse to the sg.vantagepoint.uncrackable1 package
2. **Find the check** — Locate the verification class that compares user input against a decoded secret
3. **Extract the constants** — Copy out the hardcoded AES key and the Base64-encoded ciphertext
4. **Decrypt offline** — Reproduce the AES-ECB decryption in a short Python script — no device needed

Demo — Path A: Offline Decryption Script

Reproducing the app's own (weak) crypto outside the app entirely.

```
# decrypt_offline.py — illustrative pattern, adapt constants from YOUR jadx output
from Crypto.Cipher import AES
import base64

key        = b'8d127684cbc37c17'          # extracted from CodeCheck.java
ciphertext = base64.b64decode('8ClmM+aX3AceHWyz1Rl/8A==')

cipher = AES.new(key, AES.MODE_ECB)       # matches the app's (weak) mode
plaintext = cipher.decrypt(ciphertext)
print(plaintext.rstrip(b'\x00'))          # -> the hidden secret string

# Result: the app's 'secret' is now known WITHOUT ever launching it.
```

Demo — Path B: Dynamic Instrumentation

1	**First launch fails**	Install on the rooted emulator; the app detects root and force-closes immediately
2	**Identify the checks**	In jadx, find the root/debug-detection methods the app calls before showing the UI
3	**Write a Frida hook**	Override each detection method to always report 'not rooted' / 'not debugged'
4	**Relaunch under Frida**	frida -U -f owasp.mstg.uncrackable1 -l bypass.js --no-pause
5	**Hook the verifier**	Also hook the comparison method itself to log its arguments — the secret appears in the console

Demo — Path B: Frida Hook (Illustrative Pattern)

Neutralize root detection, then log the verifier's live inputs.

```
// bypass.js — pattern only; confirm exact class/method names in YOUR jadx output
Java.perform(function () {
  // 1) Force every root/debug check in the target class to return benign values
  var checks = Java.use('sg.vantagepoint.uncrackable1.RootCheck');
  ['isRooted', 'isDebuggable', 'hasSuBinary'].forEach(function (m) {
    if (checks[m]) checks[m].implementation = function () { return false; };
  });

  // 2) Hook the verifier to print what it's comparing against, at runtime
  var verifier = Java.use('sg.vantagepoint.uncrackable1.CodeCheck');
  verifier.check_pin.implementation = function (input) {
    var result = this.check_pin(input);
    console.log('[*] user input : ' + input);
    console.log('[*] check result: ' + result);
    return result;
  };
});
```

Demo Recap: Static vs. Dynamic

Path A — Static

- No device or emulator required
- Fast, repeatable, fully offline
- Reveals the LOGIC (why it's insecure)
- Defeated by strong obfuscation / native code

Path B — Dynamic

- Requires a controlled rooted device/emulator
- Slower, but works even when code is obfuscated
- Reveals real runtime VALUES as the app computes them
- Can trip anti-instrumentation defenses (higher levels)

Common Mobile Vulnerability Classes



Insecure Storage (V2)

Secrets in SharedPreferences, SQLite, logs, clipboard, or unencrypted backups



Weak Cryptography (V3)

ECB mode, static keys/IVs, home-grown algorithms, weak RNG for tokens



Insecure Network (V5)

Cleartext traffic allowed, missing or bypassable certificate pinning



Platform Misuse (V6)

Exported components without permission checks, unsafe WebView JS bridges



Poor Code Quality (V7)

Hardcoded API keys/passwords, debuggable=true shipped to production



Weak Resiliency (V8)

Root/jailbreak and debugger detection that's trivially bypassed with Frida

Ethics, Legal & Responsible Testing

- Written authorization and defined scope BEFORE any testing — no exceptions, even for 'just curious' checks
- Only test crackmes, your own apps, or apps you have explicit permission to assess
- Handle any real secrets, credentials, or personal data you uncover with strict confidentiality
- Follow responsible/coordinated disclosure timelines when reporting real vulnerabilities
- Unauthorized access to computer systems is a criminal offense in most jurisdictions (e.g., CFAA in the US) — scope creep is not a gray area



Rule of thumb

If you would not be comfortable explaining a specific action to the app owner's legal team in writing, don't do it without asking first.

Key Takeaways & Homework

- **Mobile pentesting = web pentesting mindset + a reversible client.**
  The OS fights back with sandboxing, signing, and pinning — your job is to work around it, legally and in scope.
- **Static and dynamic analysis are complementary, not competing.**
  Use static to understand; use dynamic to confirm and to bypass runtime defenses.
- **MASVS/MASTG give you a shared vocabulary for findings.** Every issue you write up should map to a Vx control ID.
- **Homework:** Solve UnCrackable Levels 2–4 on your own; each adds obfuscation, native code, or anti-Frida tricks.



Stretch goal

Pick one MASTG test case not covered today (e.g. WebView JS-bridge abuse or exported ContentProvider SQLi) and reproduce it on a lab app.

KEEP THESE BOOKMARKED

Resources & Questions



OWASP MASTG

mas.owasp.org — full testing guide, one test case per control



OWASP MAS Crackmes

github.com/OWASP/mastestapp — UnCrackable & other practice targets



Frida Docs

frida.re — scripting API reference and platform notes



Objection

github.com/sensepost/objection — runtime mobile exploration toolkit



drozer

github.com/WithSecureLabs/drozer — Android IPC attack surface tool



MobSF

github.com/MobSF/Mobile-Security-Framework-MobSF — automated triage